Firewall Rule Optimizer using Genetic Algorithm (GA)

Student: Aiban Mohammed

1. Problem Description

Firewalls often apply a **first-match** policy: packets are checked against rules in order, and the first matching rule decides ALLOW/DENY. With large rule sets, poor ordering increases inspection time and creates rule anomalies (e.g., rules that never get used).

2. Chosen Technique

We solve the ordering problem using a **Genetic Algorithm (GA)**. Each chromosome is a complete permutation of the rule indices. The GA evolves orderings that preserve policy decisions (high coverage) while reducing average inspection cost and traffic-based shadowing.

3. Algorithm Design

Chromosome: permutation of rule indices (ordering).

Selection: tournament selection.

Crossover: Order Crossover (OX).

Mutation: swap + inversion (segment reverse).

Fitness Function

We evaluate an ordering using: **Coverage** (agreement with a reference policy on traffic samples), **Shadow rate** (rules never triggered by the traffic), and **Avg. checks** (average number of rules tested per packet). Coverage is treated as a near-hard constraint.

GA Parameter	Value
Population size	35
Generations (max)	30
Tournament k	3
Crossover rate	0.9
Swap mutation p	0.2
Inversion mutation p	0.1
Elitism	2
Min coverage	0.99

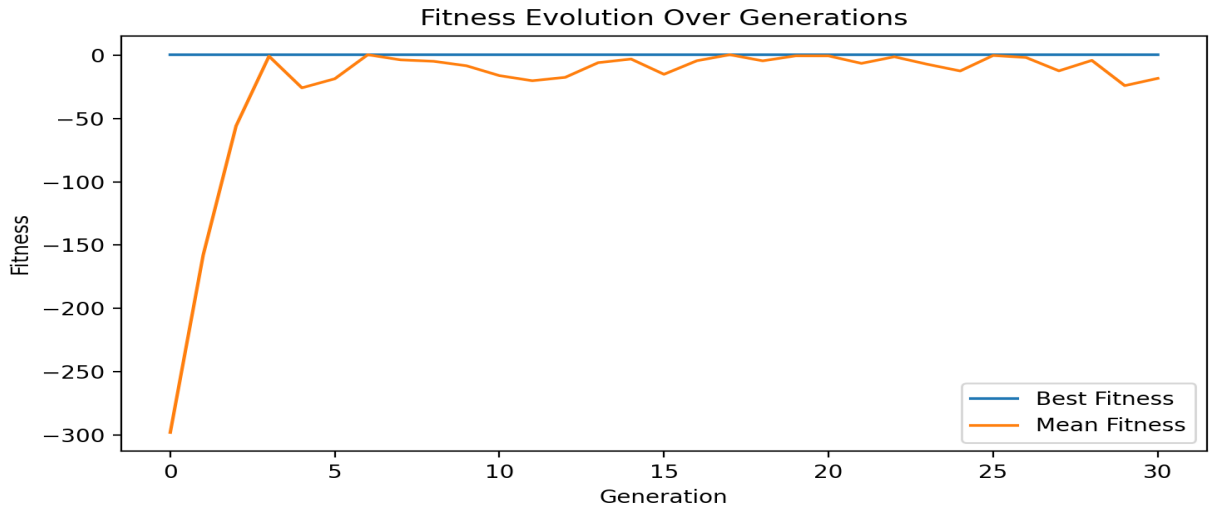
4. Experiments & Results

We compare the original (input) ordering with the GA-optimized ordering. The reference policy is the original ordering applied on the same traffic samples.

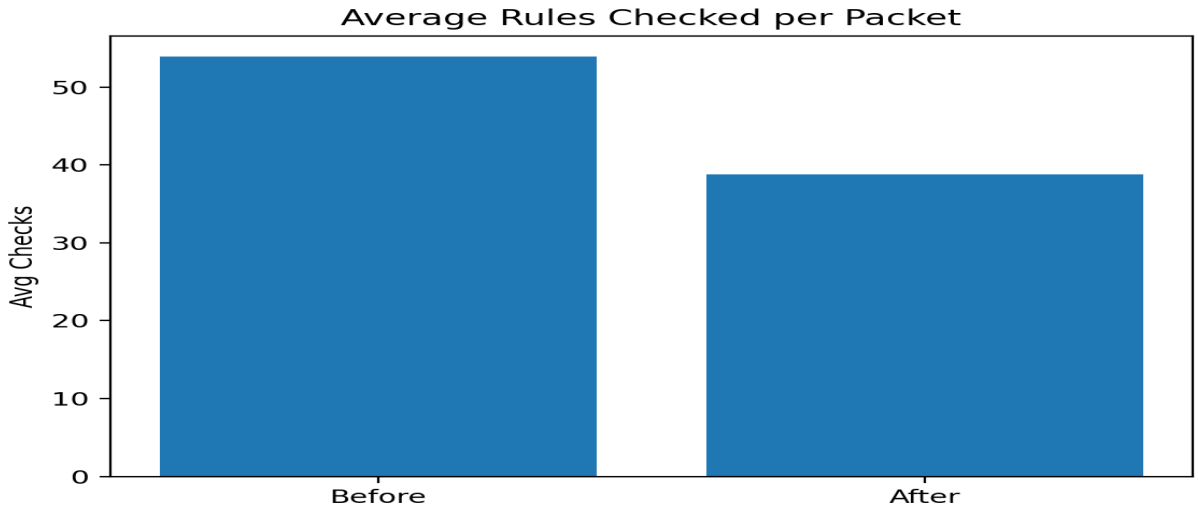
Metric	Before	After

coverage	1.0000	0.9993
avg_checks	53.8770	38.7450
shadow_rate	0.9133	0.8900
inactive_rules	274	267

Fitness evolution



Avg checks (before vs after)



5. Key Takeaways

• GA reorders rules to reduce average inspection cost while preserving policy decisions (high coverage).
• Traffic-based shadowing decreases when more specific rules are prioritized.
• Fitness improves steadily over generations; early stopping prevents over-computation.